Exercise 01:

Write a program that reads a Celsius degree and converts it to Fahrenheit.
Formula **F = (9/5) × Celsius + 32**

```
celsius = float(input("Enter a degree in Celsius: "))
fahrenheit = (9 / 5) * celsius + 32
print(f"{celsius} in Celsius is {fahrenheit} Fahrenheit")
```

Output:

```
Enter a degree in Celsius: 43
43.0 in Celsius is 109.4 Fahrenheit
```

Exercise 02:

Write a program that asks the user to enter the starting velocity v0, the ending velocity v1, and the time span t, then displays the average acceleration with two digits after the decimal point using the formula: **a = (v1 – v0) / t**

```
v0 = float(input("Enter v0: "))
v1 = float(input("Enter v1: "))
t = float(input("Enter t: "))
a = (v1 - v0) / t
print(f"The average acceleration is {a:.2f}")
```

Output:

```
Enter v0: 5.5
Enter v1: 50.9
Enter t: 4.5
The average acceleration is 10.09
```

Exercise 03:

Write a program that displays the following on the screen:

```
1 1
2 4
3 9
4 16
5 25
```

You should use the pow() function.

```
for i in range(1, 6):
    print(i, pow(i, 2))
```

Exercise 04:

Write a program that asks the user to enter the radius of a circle and then displays its area and its perimeter.
Area = π × r²
Perimeter = 2 × π × r

```
import math
radius = float(input("Enter the radius: "))
area = math.pi * pow(radius, 2)
perimeter = 2 * math.pi * radius
print("Area =", area)
print("Perimeter =", perimeter)
```

Output:

```
Enter the radius: 5
Area = 78.53981633974483
Perimeter = 31.41592653589793
```

Exercise 05:

Write a program that asks the user to enter a character and then displays its ASCII code.

```
ch = input("Enter a character: ")
print("ASCII code =", ord(ch))
```

Output:

```
Enter a character: A
ASCII code = 65
```

Exercise 06:

Given the following:
One birth every 7 seconds
One death every 13 seconds
One new immigrant every 45 seconds
Current population: 1000000
One year: 365 days
Write a program that displays the population for each of the next five years.

```
population = 1000000
births_per_year = (365 * 24 * 60 * 60) // 7
deaths_per_year = (365 * 24 * 60 * 60) // 13
immigrants_per_year = (365 * 24 * 60 * 60) // 45
for year in range(1, 6):
    population = population + births_per_year - deaths_per_year + immigrants_per_year
    print(f"Year {year}: {population}")
```

Output:

```
Year 1: 2861285
Year 2: 4722570
Year 3: 6583855
Year 4: 8445140
Year 5: 10306425
```

Exercise 07:

Write a program that asks the user to enter an integer and then displays whether it is Even or Odd.

```
n = int(input("Enter an integer: "))
match n % 2:
    case 0:
        print("Even")
    case 1 | -1:
        print("Odd")
```

Output:

```
Enter an integer: 12
Even
```

Exercise 08 (Using match):

Write a program that reads an arithmetic operation (+, -, *, or /) and two numbers x and y, then performs the operation x op y.

```
op = input("Enter an operation: ")
x = float(input("Enter x: "))
y = float(input("Enter y: "))
match op:
   case "+":
      print(f"{x} + {y} = {x + y}")
   case "-":
      print(f"{x} - {y} = {x - y}")
   case "*":
      print(f"{x} * {y} = {x * y}")
   case "/":
      print(f"{x} / {y} = {x / y}")
   case _:
      print("Invalid operation")
```

Output:

```
Enter an operation: *
Enter x: 2
Enter y: 5
2.0 * 5.0 = 10.0
```

Exercise 08 (Using if):

Write a program that reads an arithmetic operation (+, -, *, or /) and two numbers x and y, then performs the operation using if statements.

```
op = input("Enter an operation: ")
x = float(input("Enter x: "))
y = float(input("Enter y: "))
if op == "+":
   print(f"{x} + {y} = {x + y}")
elif op == "-":
   print(f"{x} - {y} = {x - y}")
elif op == "*":
   print(f"{x} * {y} = {x * y}")
elif op == "/":
   print(f"{x} / {y} = {x / y}")
else:
   print("Invalid operation")
```

Output:

```
Enter an operation: -
Enter x: 20
Enter y: 45.5
20.0 - 45.5 = -25.5
```

Exercise 09:

Write a function that computes the sum of the digits in an integer.
For example:
sum_digits(234) returns 9.
Write a test program that prompts the user to enter an integer and displays the sum of its digits.

```
def sum_digits(n):
    total = 0
    while n != 0:
        total += n % 10
        n //= 10
    return total
number = int(input("Enter an integer: "))
print("Sum of digits =", sum_digits(number))
```

Output:

```
Enter an integer: 234
Sum of digits = 9
```

Exercise 10:

A palindromic prime is a number that is both prime and palindromic.
Write a program that displays the first 100 palindromic prime numbers, displaying 10 numbers per line.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
def is_palindrome(n):
    original = n
    reverse = 0
    while n != 0:
        reverse = reverse * 10 + n % 10
        n //= 10
    return original == reverse
count = 0
number = 2
while count < 100:
    if is_prime(number) and is_palindrome(number):
        print(number, end=" ")
        count += 1
        if count % 10 == 0:
            print()
    number += 1
```

Output (First Lines):

```
2 3 5 7 11 101 131 151 181 191
313 353 373 383 727 757 787 797 919 929
```

Exercise 11:

a) Create an array of integers named list using the shorthand notation with ten values.
b) Write the code to find the sum of all elements in the array.
c) Write the code to find the highest value in the array.

```
list = [12, 5, 18, 9, 25, 7, 30, 14, 3, 20]
total = 0
for num in list:
   total += num
print("Sum =", total)
maximum = list[0]
for num in list:
   if num > maximum:
      maximum = num
print("Highest value =", maximum)
```

Output:

```
Sum = 143
Highest value = 30
```

Exercise 12:

Write methods that:
Return the average of an array.
Count the elements above a given key.
Count the elements below a given key.
Find the range (maximum − minimum).
Then write a test program that asks the user to enter ten numbers and displays:
Average
Number of values above average
Number of values below average
Range

```
def average(array):
   return sum(array) / len(array)
def count_above(array, key):
   count = 0
   for num in array:
      if num > key:
         count += 1
   return count
def count_below(array, key):
   count = 0
   for num in array:
      if num < key:
         count += 1
   return count
def find_range(array):
   return max(array) - min(array)
numbers = []
print("Enter 10 numbers:")
for i in range(10):
   numbers.append(float(input()))
avg = average(numbers)
print("Average =", avg)
print("Above Average =", count_above(numbers, avg))
print("Below Average =", count_below(numbers, avg))
print("Range =", find_range(numbers))
```

Output:

```
Enter 10 numbers:
5
7
3
10
8
4
2
6
9
1
Average = 5.5
Above Average = 5
Below Average = 5
Range = 9.0
```

Exercise 13:

Design a class named Triangle that contains:
Three sides (side1, side2, side3)
A default constructor (all sides = 1)
A constructor with three specified sides
area() method
perimeter() method
Then write a test program that creates two objects and displays their area and perimeter.

```
import math
class Triangle:
    def __init__(self, side1=1, side2=1, side3=1):
        self.side1 = side1
        self.side2 = side2
        self.side3 = side3
    def perimeter(self):
        return self.side1 + self.side2 + self.side3
    def area(self):
        s = self.perimeter() / 2
        return math.sqrt(
            s * (s - self.side1) * (s - self.side2) * (s - self.side3)
        )
t1 = Triangle()
t2 = Triangle(4, 5, 6)
print("Triangle 1")
print("Area =", t1.area())
print("Perimeter =", t1.perimeter())
print()
print("Triangle 2")
print("Area =", t2.area())
print("Perimeter =", t2.perimeter())
```

Output:

```
Triangle 1
Area = 0.4330127018922193
Perimeter = 3

Triangle 2
Area = 9.921567416492215
Perimeter = 15
```

Exercise 14:

Design a class named Account that contains:
id
balance
annualInterestRate
dateCreated
Include methods for:
Deposit
Withdraw
Monthly interest rate
Monthly interest
Then write a test program that creates an account with:
ID = 1122
Balance = 20000
Annual Interest Rate = 4.5%
Withdraw 2500, deposit 3000, then display the balance, monthly interest, and creation date.

```
from datetime import datetime
class Account:
    def __init__(self, id=0, balance=0):
        self.id = id
        self.balance = balance
        self.annual_interest_rate = 0
        self.date_created = datetime.now()
    def set_annual_interest_rate(self, rate):
        self.annual_interest_rate = rate
    def get_monthly_interest_rate(self):
        return self.annual_interest_rate / 100 / 12
    def get_monthly_interest(self):
        return self.balance * self.get_monthly_interest_rate()
    def withdraw(self, amount):
        self.balance -= amount
    def deposit(self, amount):
        self.balance += amount
account = Account(1122, 20000)
account.set_annual_interest_rate(4.5)
account.withdraw(2500)
account.deposit(3000)
print("Balance: $", account.balance)
print("Monthly Interest: $", account.get_monthly_interest())
print("Date Created:", account.date_created)
```

Output (Example):

```
Balance: $ 20500
Monthly Interest: $ 76.875
Date Created: 2026-07-26 09:00:00
Note: The date and time will vary depending on when you run the program.
```

Exercise 15:

Write your own method to check whether the first string is a substring of the second string.

```
def is_substring(s1, s2):
    return s1 in s2
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")
if is_substring(str1, str2):
    print("The first string is a substring of the second")
else:
    print("The first string is not a substring of the second")
```

Output 1:

```
Enter first string: Java
Enter second string: I love Java programming
The first string is a substring of the second
```

Output 2:

```
Enter first string: Python
Enter second string: I love Java programming
The first string is not a substring of the second
```

Exercise 16 (Inheritance):

Create a class named Person that contains:
A private string data field named name.
A private integer data field named age.
A constructor that initializes name and age.
A method display_info() that displays the person's information.
Create a subclass named Student that extends Person and contains:
A private string data field named major.
A constructor that initializes name, age, and major using super().
Override the display_info() method to display the student's information.
Write a test program that creates a Person object and a Student object and displays their information.

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age
    def display_info(self):
        print("Name:", self.__name)
        print("Age:", self.__age)
class Student(Person):
    def __init__(self, name, age, major):
        super().__init__(name, age)
        self.__major = major
    def display_info(self):
        super().display_info()
        print("Major:", self.__major)
p1 = Person("Ali", 25)
s1 = Student("Ahmad", 21, "Software Engineering")
print("Person Information:")
p1.display_info()
print()
print("Student Information:")
s1.display_info()
```

Output:

```
Person Information:
Name: Ali
Age: 25

Student Information:
Name: Ahmad
Age: 21
Major: Software Engineering
```

Exercise 17 (Polymorphism):

Write a program that demonstrates Polymorphism, Casting, and the instanceof concept by creating a list of objects and extracting only the Shape objects.

```
class GeometricObject:
    pass
class Shape(GeometricObject):
    def __init__(self, size, color, filled):
        self.size = size
        self.color = color
        self.filled = filled
    def __str__(self):
        return f"Shape(Size={self.size}, Color={self.color}, Filled={self.filled})"
def get_shapes(objects):
    shapes = []
    for obj in objects:
        if isinstance(obj, Shape):
            shapes.append(obj)
    return shapes
g = []
for i in range(0, 10, 2):
    g.append(GeometricObject())
    g.append(Shape(3 + i, "Red", False))
s = get_shapes(g)
for shape in s:
    print(shape)
```

Output:

```
Shape(Size=3, Color=Red, Filled=False)
Shape(Size=5, Color=Red, Filled=False)
Shape(Size=7, Color=Red, Filled=False)
Shape(Size=9, Color=Red, Filled=False)
Shape(Size=11, Color=Red, Filled=False)
```

Exercise 18 (List):

Write a program that creates a list of integers. The program should:
Add five numbers to the list.
Display all elements.
Remove an element.
Search for a specific number.

```
numbers = [10, 20, 30, 40, 50]
print("List:", numbers)
numbers.pop(2)
print("After removing index 2:", numbers)
n = int(input("Enter a number to search: "))
if n in numbers:
   print("Number exists")
else:
   print("Number does not exist")
```

Output:

```
List: [10, 20, 30, 40, 50]
After removing index 2: [10, 20, 40, 50]
Enter a number to search: 40
Number exists
```

Exercise 19 (Exception Handling):

Write a program that asks the user to enter two numbers and divides them. Handle possible exceptions using try-except.

```
try:
   n1 = int(input("Enter first number: "))
   n2 = int(input("Enter second number: "))
   result = n1 / n2
   print("Result =", result)
except ZeroDivisionError:
   print("Error: Cannot divide by zero")
except ValueError:
   print("Invalid input")
finally:
   print("Program finished")
```

Output 1:

```
Enter first number: 20
Enter second number: 5
Result = 4.0
Program finished
```

Output 2:

```
Enter first number: 20
Enter second number: 0
Error: Cannot divide by zero
Program finished
```

Exercise 20 (Text I/O and File Handling):

Write a program that creates a text file named students.txt, writes student names into the file, then reads and displays the file contents.

```
try:
    with open("students.txt", "w") as writer:
        writer.write("Ali\n")
        writer.write("Ahmad\n")
        writer.write("Sara\n")
    print("File created successfully")
except IOError:
    print("Error writing file")
try:
    with open("students.txt", "r") as reader:
        print("Students:")
        for name in reader:
            print(name.strip())
except FileNotFoundError:
    print("File not found")
```

Output:

```
File created successfully
Students:
Ali
Ahmad
Sara
```

Exercise 21 (Enum) :

Create an enum named Day that contains the days of the week.
Write a program that asks the user to enter a day and displays whether it is a Weekday or Weekend.

```
from enum import Enum
class Day(Enum):
    MONDAY = 1
    TUESDAY = 2
    WEDNESDAY = 3
    THURSDAY = 4
    FRIDAY = 5
    SATURDAY = 6
    SUNDAY = 7
day_input = input("Enter a day: ").upper()
day = Day[day_input]
if day in (Day.SATURDAY, Day.SUNDAY):
    print("Weekend")
else:
    print("Weekday")
```

Output 1:

```
Enter a day: Monday
Weekday
```

Output 2:

```
Enter a day: Sunday
Weekend
```

Exercise 22 (Linked List - Anagram):

Two words are anagrams if one word can be formed by rearranging the letters of the other. Write a method that receives two linked lists representing two words and returns True if they are anagrams; otherwise, return False.

```
from collections import deque
def is_anagram(w1, w2):
    if len(w1) != len(w2):
        return False
    temp = deque(w2)
    for ch in w1:
        if ch in temp:
            temp.remove(ch)
        else:
            return False
    return True
word1 = deque("treason")
word2 = deque("senator")
print(is_anagram(word1, word2))
```

Output:

```
True
```

Exercise 23 (Linked List - Sum of Values):

Create a linked list containing the values: 6, 4, 10, 9, 1
Write a method sum_values() that returns the sum of all values.

```
from collections import deque
class MyLinkedList:
    def __init__(self):
        self.list = deque()
    def sum_values(self):
        total = 0
        for value in self.list:
            total += value
        return total
obj = MyLinkedList()
obj.list.append(6)
obj.list.append(4)
obj.list.append(10)
obj.list.append(9)
obj.list.append(1)
print("Sum =", obj.sum_values())
```

Output:

```
Sum = 30
```

Exercise 24 (Linked List Parameter):

Write a function sum_numbers() that receives a linked list as a parameter and returns the sum of its values.

```
from collections import deque
def sum_numbers(lst):
    total = 0
    for number in lst:
        total += number
    return total
numbers = deque([6, 4, 10, 9, 1])
print("Sum =", sum_numbers(numbers))
```

Output:

```
Sum = 30
```

Exercise 25 (Stack - Balanced Parentheses):

Write a function check(expression) that checks whether the parentheses in an expression are balanced using a Stack.

Write a test program to demonstrate the function.

```
def check(expression):
    stack = []
    for ch in expression:
        if ch == "(":
            stack.append(ch)
        elif ch == ")":
            if not stack:
                return False
            stack.pop()
    return len(stack) == 0
expression = input("Enter expression: ")
if check(expression):
    print("Balanced expression")
else:
    print("Not balanced expression")
```

Output 1:

```
Enter expression: (((a+b)*c)+(u*v))
Balanced expression
```

Output 2:

```
Enter expression: (a+b)*c)
Not balanced expression
```

Exercise 26 (Queue):

In a bank, customers wait in a queue (q0) for their turn to complete a transaction.
Write a method split_queue(q0, q1, q2) that splits the customers into two queues:
q1: Names starting with A–H
q2: Names starting with I–Z
Write a test program that reads customer names, fills q0, calls the method, and displays the contents of q1 and q2.

```
from collections import deque
def split_queue(q0, q1, q2):
    while q0:
        name = q0.popleft()
        first_letter = name[0].upper()
        if "A" <= first_letter <= "H":
            q1.append(name)
        else:
            q2.append(name)
q0 = deque()
q1 = deque()
q2 = deque()
n = int(input("Enter number of customers: "))
for i in range(n):
    name = input("Enter customer name: ")
    q0.append(name)
split_queue(q0, q1, q2)
print("\nQueue 1 (A-H):")
while q1:
    print(q1.popleft())
print("\nQueue 2 (I-Z):")
while q2:
    print(q2.popleft())
```

Output:

```
Enter number of customers: 6
Enter customer name: Ali
Enter customer name: Sara
Enter customer name: Hassan
Enter customer name: Omar
Enter customer name: Ahmad
Enter customer name: Lina

Queue 1 (A-H):
Ali
Hassan
Ahmad

Queue 2 (I-Z):
Sara
Omar
Lina
```

Exercise 27 (Big-O):

Prove that: $n^3 + 1000 = O(n^3)$

According to the definition of Big-O:

$f(n) = n^3 + 1000$ $g(n) = n^3$

We need constants c and n_o such that: $f(n) \leq c \times g(n)$

Choose: $c = 2$ $n_o = 10$ For every $n \geq 10$:

$n^3 \geq 1000$

Therefore: $n^3 + 1000 \leq n^3 + n^3$

So: $n^3 + 1000 \leq 2n^3$

Hence: $f(n) \leq c \times g(n)$

Therefore: $n^3 + 1000 = O(n^3)$

Final Answer: $c = 2$ $n_o = 10$

Exercise 28 (Big-O Analysis):

Determine the running time of the following code using Big-O notation.

Step 1: Create the array int[] a = new int[n] --> O(1)

Step 2: First loop for (i = 0; i < n; i++) --> O(n)

Step 3: Second loop for (i = 0; i < n; i++) --> O(n)

Step 4: Total Running Time O(1) . O(n) . O(n) = $O(n^2)$

Exercise 29 (Recursion):

Write a recursive method named odd_sum() that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example:

odd_sum(5) = 1 + 3 + 5 = 9

```
def odd_sum(n):
    if n == 1:
        return 1
    return n + odd_sum(n - 2)
n = int(input("Enter an odd number: "))
print("Sum =", odd_sum(n))
```

Output:

```
Enter an odd number: 7
Sum = 16
```

Exercise 30 (Recursive oddSum) :

Write a recursive method named odd_sum() that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example:

odd_sum(7) = 1 + 3 + 5 + 7 = 16

```
def odd_sum(n):
    print("odd_sum(", n, ")", sep="")
    if n == 1:
        return 1
    return n + odd_sum(n - 2)
n = int(input("Enter a positive odd number: "))
print("Sum of odd numbers =", odd_sum(n))
```

Output:

```
Enter a positive odd number: 9
odd_sum(9)
odd_sum(7)
odd_sum(5)
odd_sum(3)
odd_sum(1)
Sum of odd numbers = 25
```

Exercise 31 (Recursion - Palindrome):

A palindrome is a word that reads the same forwards and backwards.
Write a recursive method named is_palindrome() that takes a string as a parameter and returns True if the word is a palindrome and False otherwise.
Write a test program to test the method.

```
def is_palindrome(word):
    if len(word) <= 1:
        return True
    if word[0] != word[-1]:
        return False
    return is_palindrome(word[1:-1])
word = input("Enter a word: ")
if is_palindrome(word):
    print("The word is a palindrome")
else:
    print("The word is not a palindrome")
```

Output 1:

```
Enter a word: otto
The word is a palindrome
```

Output 2:

```
Enter a word: hello
The word is not a palindrome
```

Exercise 32 (Tree & Recursion):

Write a recursive method that checks whether a binary tree is a full binary tree.
A full binary tree is a tree in which every node has either 0 or 2 children.

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
def is_full_binary_tree(root):
    if root is None:
        return True
    if root.left is None and root.right is None:
        return True
    if root.left is None or root.right is None:
        return False
    return (is_full_binary_tree(root.left) and
            is_full_binary_tree(root.right))
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
root.right.left = TreeNode(6)
root.right.right = TreeNode(7)
if is_full_binary_tree(root):
    print("The tree is a full binary tree")
else:
    print("The tree is not a full binary tree")
```

Output:

```
The tree is a full binary tree
```

Example of NOT Full Binary Tree:

```
   1
  / \
 2   3
/
4
```

Output:

```
The tree is not a full binary tree
```

Exercise 33 (Graphs - DFS):

Create a graph using an adjacency list and implement Depth First Search (DFS) to visit all vertices.

```
class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adjacency_list = [[] for _ in range(vertices)]
    def add_edge(self, source, destination):
        self.adjacency_list[source].append(destination)
    def dfs(self, start):
        visited = [False] * self.vertices
        self._dfs_util(start, visited)
    def _dfs_util(self, vertex, visited):
        visited[vertex] = True
        print(vertex, end=" ")
        for neighbor in self.adjacency_list[vertex]:
            if not visited[neighbor]:
                self._dfs_util(neighbor, visited)
graph = Graph(5)
graph.add_edge(0, 1)
graph.add_edge(0, 2)
graph.add_edge(1, 3)
graph.add_edge(2, 4)
print("DFS Traversal:")
graph.dfs(0)
```

Output:

```
DFS Traversal:
0 1 3 2 4
```

Exercise 34 (Dictionary):

Create a program that stores student names and their grades using a dictionary, then searches for a student's grade.

```
students = {
    "Ali": 90,
    "Ahmad": 85,
    "Sara": 95
}
print("Students Grades:")
print(students)
name = input("Enter student name: ")
if name in students:
    print("Grade =", students[name])
else:
    print("Student not found")
```

Output:

```
Students Grades:
{'Ali': 90, 'Ahmad': 85, 'Sara': 95}
Enter student name: Sara
Grade = 95
```

Exercise 35 (List):

Create a list that stores numbers. Add elements, remove an element, and display the list.

```
numbers = []
numbers.append(10)
numbers.append(20)
numbers.append(30)
numbers.append(40)
print("List:", numbers)
numbers.pop(2)
print("After removing element:", numbers)
print("First element:", numbers[0])
```

Output:

```
List: [10, 20, 30, 40]
After removing element: [10, 20, 40]
First element: 10
```